



Towards a Reference Architecture for Model-Based Engineering Environments

Sebastian J. I. Herzig*, Robert Karban*
and Michel D. Ingham†

Jet Propulsion Laboratory, California Institute of Technology, Pasadena, California, USA

A key aspect of adopting model-based systems engineering as a practice in an organization is the design and development, and adoption of corresponding processes and tools that support the model-based paradigm. In an effort to enable the unified implementation of such processes and tools, this paper introduces a reference architecture model that serves as a specification for a model-based engineering environment. Current systems engineering practices, products, processes and technologies are used as input for continuously refining the architecture model. In the paper, we introduce and report on the current status of this reference architecture model, and present the methodology applied in developing the reference architecture. We conclude that while there are a very large number of domain- or application-specific processes requiring specialized behavior, these can be reduced through abstraction to a small set of core functions that need to be supported by a realization of a model-based engineering environment. Only very few organization-, domain- or application-specific aspects require specialized consideration.

I. Introduction

OVER the past 10 years, Model-based Systems Engineering (MBSE) has seen widespread industrial acceptance. The value proposition of using formal models in the design and development of complex systems for the purpose of reuse, automation, and more streamlined development processes has been demonstrated and reported in a variety of contexts and organizations through a number of pilot efforts. However, due to the lack of appropriate standards and guidelines for tooling, and tool- and model-interchange, as well as the lack of well-founded modeling languages and proven model-based design methods, there is currently a broad landscape of incompatible modeling infrastructure implementations, each solving an application- or domain-specific (oftentimes strongly overlapping) set of concerns. In a broader application context, this can lead to a lack of alignment of parallel related product development efforts (e.g., *Mass Equipment List* and *Power Equipment List*), and results in inconsistent support for users of current MBSE infrastructures in the context of large-scale projects. To address this concern, NASA's Jet Propulsion Laboratory (JPL) is developing, within the context of the Integrated Model-Centric Engineering (IMCE) effort, a reference architecture¹ for specifying and implementing a (project- or organization-) specific realization of a *Model-based Engineering Environment* (MBEE). The primary contribution of this paper is to introduce the scope and purpose of this effort, motivate its importance, describe our modeling approach, and report on the status of the reference architecture.

Even though a number of organizations have adopted MBSE practice, little to no work in specifying a coherent, general underlying modeling infrastructure has been reported in the related literature. The reference architecture introduced in this paper fills this gap by providing an architectural description of a broad set of realizations of a MBEE ecosystem, and includes in its description an appropriate set of system design and development lifecycle tools and models, and how they should be related and integrated. It also captures interactions with users in its environment and expected internal responses to these. Primarily, the focus of the work to date has been on eliciting the required functionality of a MBEE to support systems

*Software Systems Engineer, Flight Systems Engineering, Integration & Test, 4800 Oak Grove Dr., AIAA Member.

†Software Systems Engineer, Project Systems Engineering & Formulation, 4800 Oak Grove Dr., AIAA Associate Fellow.

engineering practice. The reference architecture model is intended to be used as a specification to guide development of a concrete implementation of a MBEE.

The paper begins with an overview of the reference architecture effort in section II, where the primary focus is on describing the purpose, scope, and roles of various consumers of the reference architecture. Thereafter, section III introduces the developed reference architecture model and our development approach. This is followed by a description of the current status of the reference architecture model in section IV. Towards the end of the paper in section V, we compare our effort to other, related efforts and describe how these can (and do) benefit from our work. The paper closes with conclusions drawn to date and recommendations for future work in section VI.

II. Reference Architecture Overview

There are a number of ways in which a reference architecture for a MBEE can be utilized. It is expected that various stakeholders will consume it in different ways. This section presents the purpose and scope of the reference architecture.

A. Purpose & Scope

A MBEE provides basic services for constructing, modifying, storing, managing, transforming, analyzing, visualizing and integrating system models. It is the centerpiece of a larger MBEE ecosystem which provides the necessary functionality for supporting common systems engineering practices and processes. The reference architecture introduced in this paper is primarily a specification and architectural description of such a MBEE and how it is embedded in its ecosystem. Included are the specification of an appropriate set of system development lifecycle tools that support, e.g., system architecture development, requirements elicitation, and quality and risk assessment. Captured is the IMCE perspective on how these should be integrated. The reference architecture also captures the type of information flow that occurs within a MBEE.

The *purpose* of the reference architecture introduced in this paper is to:

- Unify the MBSE infrastructure and architecture that is shared across projects and various MBSE efforts both internal and external to JPL
- Create an alignment of the related parallel product development efforts
- Standardize and provide consistent support for users of a MBSE infrastructure deployed and / or developed in an organization

The reference architecture is intended to be used as a specification to guide development of a concrete implementation of the full (or a part of a) MBEE. Internally, it is also intended to give an overview of the developed MBSE capabilities at JPL that are either currently being used, under developed, or being investigated or recommended for implementation in the future.

The intended *scope* of the reference architecture includes:

- Specification of the functionality of tooling support for domain-specific modeling, UML / SysML embeddings using profiles, modeling patterns, and repositories
- Description of a unified integration mechanism with other engineering models and tools, transformation and analysis mechanisms
- Documentation of adapted systems engineering processes that make use of key advantages of model-based systems engineering practices (such as model transformations, reuse and automation)
- Specification of a development environment for implementing or extending a MBEE
- Reusable libraries (e.g., models, practices)

B. Intended Consumers

There are several categories of stakeholders who form the set of intended consumers of the reference architecture:

- Users of a MBEE
- Developers of a MBEE
- Developers of systems interacting with a realized MBEE
- Organizations and project management

Users of a MBEE will primarily use the reference architecture to inform themselves about standard MBSE practices and processes, as well as the possible and intended interactions with various tools in a MBEE infrastructure. For this purpose, the reference architecture captures a number of operational scenarios which describe how users interact with a MBEE. Thereby, the reference architecture also captures the functions supported by the MBEE. Given that the operational scenarios are intended to guide users in how to interact with a MBEE, and capture what products are produced as part of certain processes, the reference architecture also captures rigorous MBSE practice.

A second major class of stakeholders is the architects and developers of a realization architecture specification, and the implementers of such a specification to realize a specific MBEE. Likely to be of primary interest to this class of stakeholders are the specified interfaces of a MBEE (including user interfaces), the functional components, how these are integrated to support key driving operational scenarios, how these components and integrations can be realized using off-the-shelf software components, as well as specifications for reference implementations to implement certain specified functionality.

Some stakeholders develop systems external to a MBEE, which are meant to interface with a realized and possibly already deployed MBEE. This class of stakeholders is related to the class of architects and developers of a realization specification of a MBEE, but consume the reference architecture differently. Systems external to a MBEE are typically not an integral part of a software infrastructure intended to support model-based design and development, but may be required due to the specific needs of a project, required within a certain application domain, or required to support standard practices of an institution or organization. For instance, a physical test bed for testing developed hardware components falls into this category. To satisfy the needs of this class of stakeholders, the reference architecture intends to describe how the MBEE interfaces with other external components.

Finally, the intended use of the reference architecture for stakeholders that fall into the category of organizational and project management is the identification of gaps in applied and documented practice. This allows for existing processes to be streamlined and practices to evolve.

III. Reference Architecture Model Development and Organization

As part of the overall reference architecture development effort, three primary artifacts are produced: a SysML² model of the reference architecture, an accompanying architectural design description (ADD) document, and a specification of a particular realization. All information is stored in the SysML model, and the accompanying documentation is generated using OpenMBEE.³ In this section, the SysML model describing the reference architecture is introduced. The latter two products (the ADD and realization specification) are considered outside the scope of this paper.

A. Operational Domain

The reference architecture model consists of three main layers: the operational domain, the ecosystem, and the system of interest.

The operational domain defines the overall scope of the MBEE system. It includes the MBEE ecosystem, the set of stakeholders interacting with the MBEE ecosystem (such as users) as well as any external entities or systems relevant to the context or impacting a MBEE (such as organization-specific rules). The structural definition of the operational domain is illustrated in figure 1.

The MBEE *ecosystem* includes all components that are necessary to satisfy common systems engineering functions and practices within an organization. The ecosystem includes the system of interest (i.e., the

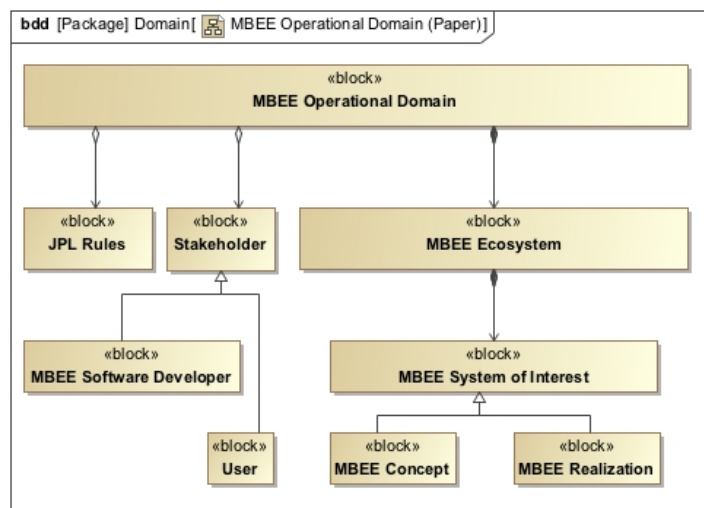


Figure 1. MBEE Operational Domain.

specification of a MBEE itself) as well as other (external) systems which may be required per organizational practice or are a result of designing and developing a system using a MBEE. For instance, elements in the ecosystem can include test beds and a repository of practices.

The system of interest is the specification of a MBEE, which includes a specification of the software components that are to be implemented, deployed and provided to a set of users. The criteria for a particular component being inside or outside the MBEE system of interest is whether we can impose requirements on it or not, and whether it represents a component that is an essential basis for performing MBSE practice.

B. Development Approach

Overall, the development of the reference architecture model is guided by the *Object-Oriented Systems Engineering Method* (OOSEM).⁴ OOSEM suggests a scenario-driven development approach. For this purpose, we capture typical systems engineering and modeling-related activities as SysML Use Cases, which are then elaborated as operational scenarios using SysML Activity diagrams. These operational scenarios capture the actions and decision processes that various entities in the domain and ecosystem go through to reach a particular state (e.g., in which a particular product has been produced) in response to certain conditions being fulfilled. Operational scenarios specify interactions with, and expected responses from a MBEE that are necessary in order to support the activities performed by stakeholders and other external entities.

As suggested in Figure 1, the system of interest is described from three perspectives: 1) system-level interactions (“black box” perspective); 2) functional interactions in the system (“conceptual” perspective); and 3) specific interactions in an implemented system (“realization” perspective). Operational scenarios are defined at the “black box” level and are used to elicit interactions of users and external systems with a MBEE. Thereby, the black box specification defines the system’s externally observable behavior and physical characteristics. It does not specify how the system achieves this externally observable behavior.⁴ Instead, this is done from the other two perspectives. The “MBEE Concept” is a refinement of the black box view on the system from a functional perspective. That is, how the externally observable behavior is achieved is specified from a functional (that is, implementation- and technology-agnostic) perspective. Similarly, the “MBEE Realization” is representative of one of several possible realizations of the black box specification in which implementation-specific choices are explicitly captured.

The set of use cases and operational scenarios is elicited in interviews with representatives from a relevant discipline or organization that is typically engaged in a (JPL) systems engineering effort (electrical systems engineering, verification & validation, etc.). Use cases are selected based on their scope and expected coverage of common systems engineering activities. Each of the use cases is used as a basis for an operational scenario, in which the interactions between different stakeholders and the MBEE ecosystem are detailed. If,

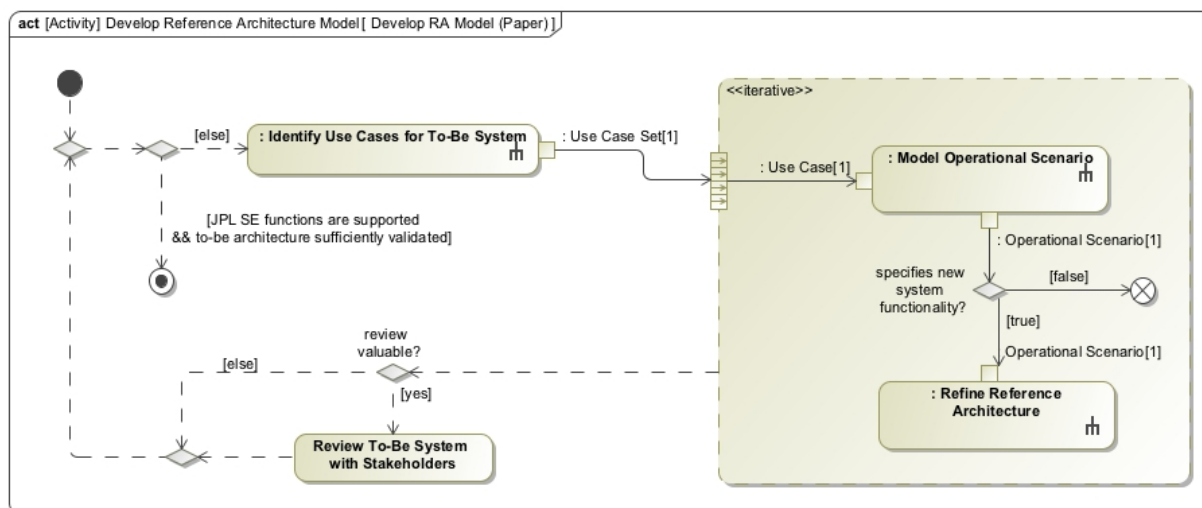


Figure 2. Scenario-driven development: using operational scenarios as driving use cases for validation & refinement of the reference architecture.

in the process of modeling this operational scenario, new and unsupported required behavior of the MBEE infrastructure is discovered, the reference architecture is refined. Otherwise, the next use case is considered. This process is depicted in Figure 2 and can be thought of as a continuous cycle of validation and subsequent refinement.

Once a set of related use cases that has been elicited from a particular discipline or organization is modeled in sufficient detail, and the reference architecture model has been refined accordingly, a review is held. In this review, representatives of the relevant discipline or organization are asked to critically evaluate the envisioned functionality and behavior of the MBEE. If rework is required, the same use cases are used in a refined form in a following cycle to further refine the reference architecture.

C. Representative Driving Use Case: Model Validation

To illustrate how the reference architecture model is refined, a representative use case and operational scenario are introduced in the following. The use case being introduced is *Validate Models* - that is, checking the internal consistency and correctness of models.⁵

The use case is elaborated as an operational scenario using SysML Activity diagrams, first for the “black box” view, then for the “conceptual” view. This is illustrated in figure 3 for the *Validate Models* (or, alternatively, *Audit Models*) use case. An important part of any operational scenario are the definitions of the state of the world prior to and post the execution of the specified events. In the case of the *Validate Models* use case, it is assumed as a pre-condition that some model data has been created that can be validated using the specified framework. After completing the scenario, it is assumed that zero or more validation results and, if applicable, suggested fixes are presented to the user. Note that the context for the use case is the *MBEE Operational Domain* (see Figure 1). The swimlanes represent elements of the domain that are participants in the use case. Here, these are: a particular user of the infrastructure (specifically, a *Systems Engineer*); and the *MBEE System of Interest* embedded in the *MBEE Ecosystem*. In the scenario, the user first defines the scope of validation: this may be the entirety of a (collection of) model(s), or a part of a model. The MBEE is then specified to extract the relevant parts of the model(s). Once the user initiates a request for validating the models, the MBEE internally runs a set of validation procedures on the selected scope. Once finished, any validation errors and suggested fixes are returned to the participating user.

In the operational scenario, the behavior of each action performed by the MBEE System of Interest is defined by a particular corresponding SysML Activity, which is part of the set of *owned behaviors* of the MBEE System of Interest. This containment of the activities by the MBEE System of Interest is illustrated in Figure 4 using the activity “Validate Model” as an illustrative example. Note that, in our approach,

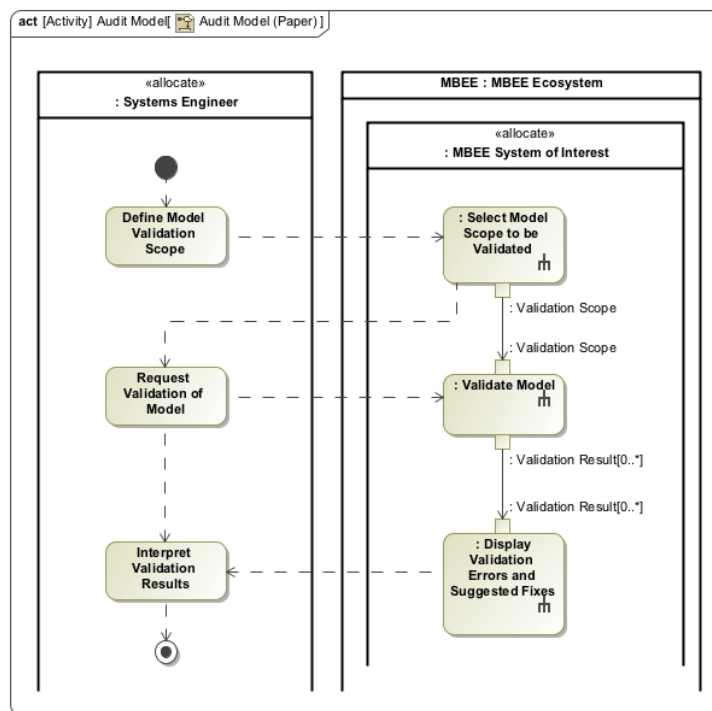


Figure 3. Sample driving scenario: validation of systems engineering models.

we also define an equivalent *Operation* whose method is defined by the corresponding SysML Activity (see *operations* compartment in Figure 4).

The reference architecture model is further refined by elaborating each action that is specified to be performed by the MBEE System of Interest in the context of the “MBEE Concept” - that is, in the context of the functional architecture description. For this purpose, the inherited behavior (in this case “Validate Model”) is first redefined in the context of “MBEE Concept”, and marked as a specialization of the corre-

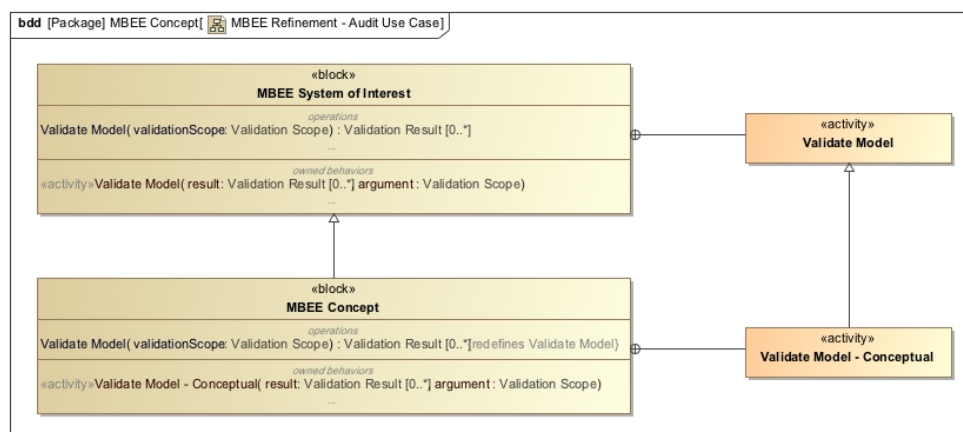


Figure 4. Illustration of how “Validate Model”, which is defined as a top-level behavior of the system of interest (see Figure 2), is related to its refinement within the context of the functional elaboration of the system of interest. Note that the owned behaviors are the methods of the Operations.

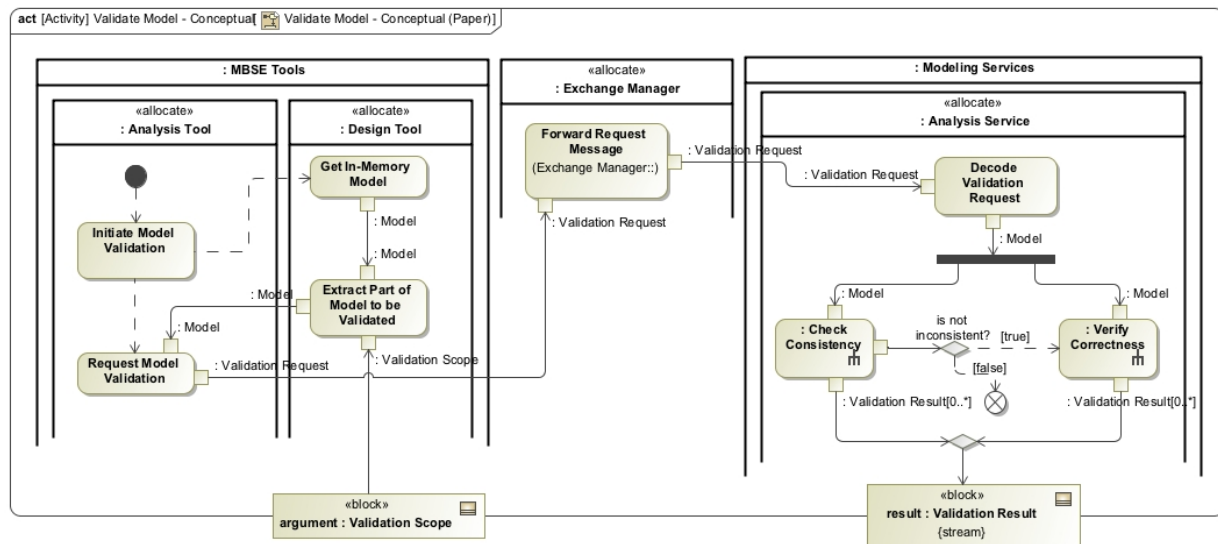


Figure 5. Functional elaboration of the activity “Validate Model”. Note that functional aggregates (components that have related functionality) are shown in the swimlanes, and the illustrated actions are allocated to these components.

sponding behavior owned by the black box description of the system. Similarly, the inherited Operation is redefined. As a convention, the same name is used for the redefined behavior with “Conceptual” as a postfix. This modeling pattern is illustrated in Figure 4.

Each of the redefined behaviors is now elaborated in the context of the functional architecture model. Similar to defining operational scenarios, SysML Activity diagrams are used in the process of elaboration. This is illustrated in Figure 5 for the case of the action “Validate Model” described in the operational scenario

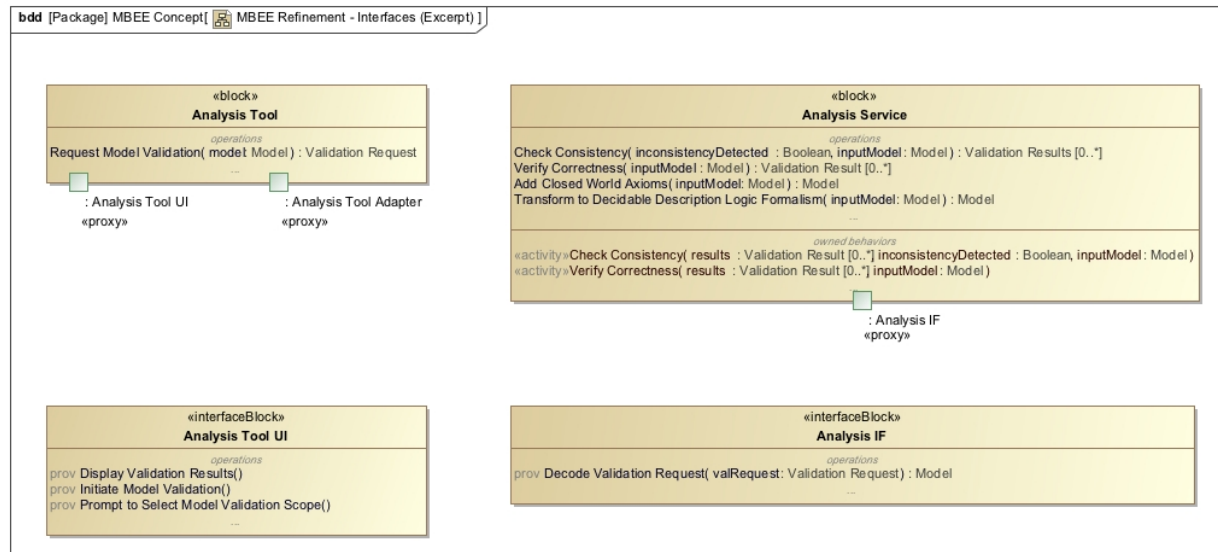


Figure 6. Excerpt of definitions of the behavior of (or, functions performed by) the components and their interfaces. Note that the methods of the Operations are defined by corresponding activities owned by the blocks.

in Figure 3. There, validation is specified to be initiated by retrieving the previously selected model scope, and passing it on to a backbone service (“Analysis Service”) that validates the models in two steps: first, the consistency of the models is checked (to determine impossible states or contradicting assertions) and, if deemed consistent, a number of correctness checks are executed (to identify violations of, e.g., organizational heuristics or conventions). An “Exchange Manager” is specified as the entity responsible for communication and data interchange.

Similar to the definition of the “MBEE System of Interest”, actions performed by functional components are captured as SysML Operations and, if further elaborated, as owned behaviors using SysML Activity diagrams. The type of data flows and information exchanges occurring between participants in the operational scenarios is captured in the reference architecture model using SysML Ports and SysML Interface Blocks. These define 1) what type of data flows between components and 2) which Operations are provided or required by individual components or interfaces. Figure 6 illustrates this for the case of the “Analysis Tool” and “Analysis Service” components. Note that “Analysis Tool” is defined to have two interfaces: a user interface (“Analysis Tool UI”) and an interface capturing any inter-tool communication (“Analysis Tool Adapter” (not illustrated in figure)). This differentiation is made to separate concerns as much as possible. “Analysis Service” has one interface (“Analysis IF”) used primarily for receiving requests from other components in the infrastructure.

Note that the operational scenario and functional elaborations of the activities do not make any assumptions about 1) how the data is captured, and what formalism or modeling language is used, and 2) what technologies are used. This leads to a description that is general in that it is free of any technological choices, yet specific enough to allow for technologies or formalisms to be identified that are capable of fulfilling the specified functionality.

IV. Status of the Reference Architecture Model

To date, the reference architecture model has been refined using both discipline-specific and foundational use cases. Discipline-specific use cases capture operational scenarios common to typical systems engineering activities. To date, mostly scenarios related to *Verification & Validation* and *Electrical Systems Engineering* have been considered. Foundational use cases consider elemental model management scenarios, such as versioning or validation of models. Each of these operational scenarios has led to additional refinements of the functional architecture.

A. Use Case Coverage

While use cases from a number of discipline areas have been elicited through interviews with domain experts, a subset of these identified use cases related to verification & validation, electrical systems engineering and model management have been fully elaborated and used as driving use cases for refining the reference architecture model. Table 1 summarizes the number of use cases that were identified and elaborated in each use case area.

We acknowledge the fact that it is impossible to cover all possible scenarios using finite resources. Our goal is to base our refinement on a set of use cases that cover a broad spectrum of the general tasks performed by systems engineers. This requires judgment by domain experts when prioritizing which use cases to elaborate. Based on our current results and findings, we believe that there is a (limited) set of core capabilities that

Table 1. Number of identified and elaborated driving use cases for various domain- and discipline-specific concerns related to design & development, and modeling & model management related concerns.

Concern Area	# Identified	# Elaborated
Verification & Validation	11	11
Electrical System Engineering	10	8
Flight System Engineering	2	2
Behavior Modeling	6	1
Model Management	25	7

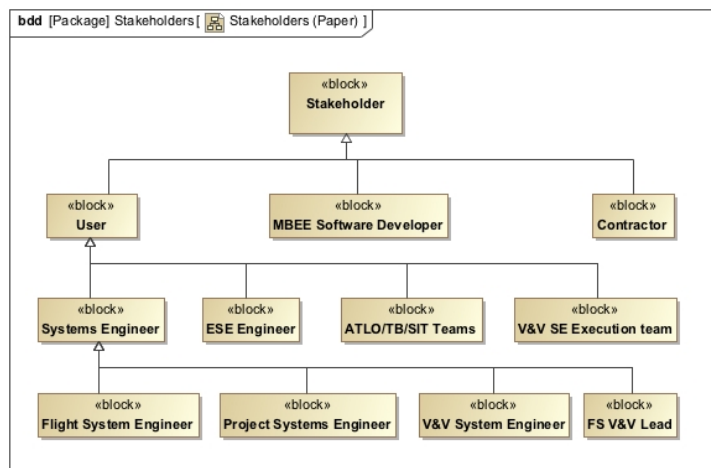


Figure 7. Excerpt of the stakeholder hierarchy.

any implementation of a MBEE should fulfill in order to support most model-based systems engineering processes. This set can be derived by generalizing from the specific functionality captured in the operational scenarios.

B. Completeness of the Operational Domain and Ecosystem

In the process of capturing operational scenarios, a number of stakeholders (such as users of the MBEE) and other external entities have been identified. A representative excerpt of the hierarchy of stakeholders resulting from this effort is captured in Figure 7. The full hierarchy is not shown for brevity. Note the

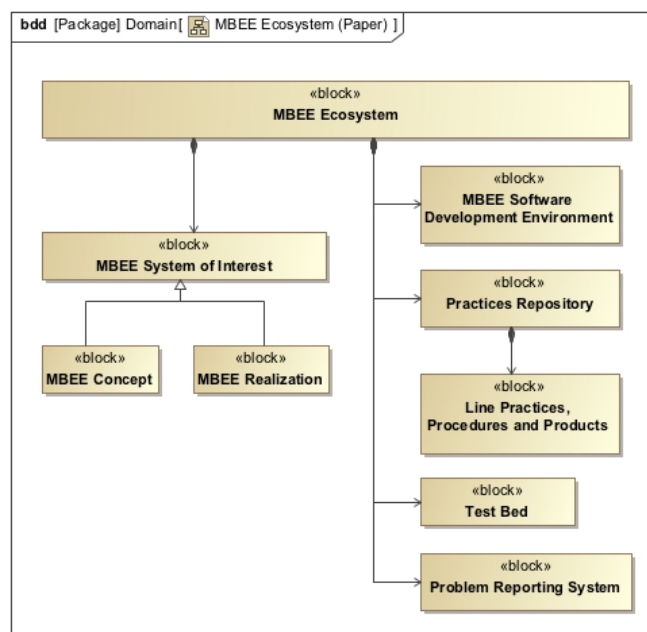


Figure 8. MBEE Operational Domain.

relation of this hierarchy to the operational domain depicted in Figure 1.

In addition to stakeholders in the domain, a number of entities that are part of the ecosystem have been identified. These are captured in Figure 8 and include a *Test Bed* and the JPL *Problem Reporting System*. A Test Bed was identified as a necessary element in select verification & validation scenarios in which hardware is tested to determine whether certain requirements are met. The Problem Reporting System is used in a similar context: it is an institutional tool used for capturing anomalies during the execution of tests.

It is expected that the operational domain definition will continue to grow as more use cases are covered. However, at a certain level of abstraction, this definition is already deemed complete: for instance, while there are systems engineers focused on activities within a certain discipline (e.g., a “V & V Systems Engineer”), the more general role of a systems engineer has been captured, from which a number of generic interactions with the MBEE can be derived.

C. Functional Architecture

As part of the development of the reference architecture model, a functional decomposition of the system of interest was created. This decomposition of the functional architecture (i.e., the structural decomposition of “MBEE Concept”) is illustrated in Figure 9. The initial breakdown of the functional architecture into the illustrated components was inspired by the ModelBus architecture⁶ and was further refined by analyzing current (MB)SE practice at JPL. The integration of operational scenarios aided in further maturing the functional decomposition.

The components of the functional architecture are grouped into six primary functional units:

- MBSE Tools
- Process Tools
- Core Services
- Modeling Services
- Other Services
- Exchange Manager

The conceptual components are specified to exchange information across a *Exchange Manager*. *Tools* represent the front end (GUI oriented applications) whereas *Services* represent back-end applications that require no user interaction. The latter is primarily meant to represent applications that perform expensive computations or analyses, while the former represents the agglomeration of tools that prepare the input to, and present the output of, such computationally expensive processes. In a realization specification of a MBEE, multiple realization elements may implement the functionality specified by one or more conceptual elements (at least partially). For instance, certain SysML modeling tools can realize most functionality specified by the “Design Tool” and “Requirements Tool”. However, not all desired functionality may be realized in a given tool.

As illustrated in Figure 9, each functional component has an interface defined by a SysML Port and the corresponding SysML Interface Block definition. Communication paths are illustrated using SysML Connectors. As an example, consider the “Analysis Tool” and its ports, as well as the “Analysis Service”. The definitions of the components were already introduced in Figure 6. Types of information exchanged across tools and services is captured in a data model, consisting of types and their associative relations. Note that the reference architecture defines multiple, related variants of such data models. In the black box and functional specification, these types represent the kind of information being exchanged, thereby being agnostic of the formal (modeling) language it is represented in. In this form, the data model being used in this context resembles an ontology. Each realization specification defines its own implementation-specific extension of this data model, where specific encodings of the information and knowledge being exchanged are made explicit. For instance, consider the type of information being exchanged in Figure 5: here, “Model”, “Validation Result” and “Validation Scope” represent abstract concepts. In a realization specification, “Model” may be a “SysML Model of Spacecraft XYZ” or an “AADL Model of Spacecraft XYZ’s Avionics System”. Similarly, “Validation Result” may be specified as an instance of a domain-specific modeling language, or be a textual document with a specific structure.

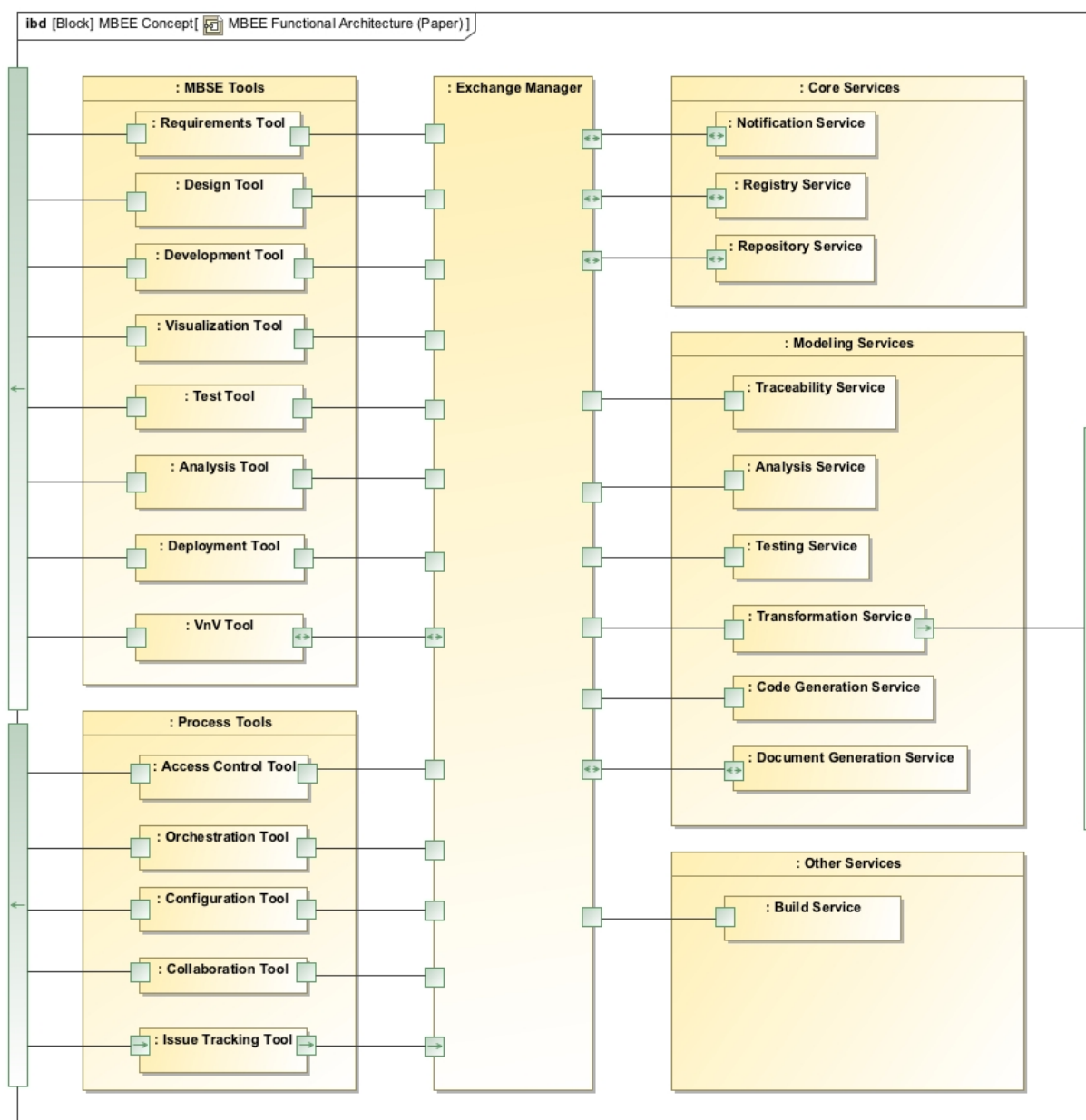


Figure 9. Decomposition of the MBEE into functionally related components. Ports and connectors are used for describing data flows.

Given definitions of the interfaces, and a description of the driving operational scenarios, all elements required for specifying a well-defined system architecture are given. Interfaces define what information is supplied to, and provided (or produced) by components. The definition of the behavior of the functional elements defines how various inputs are transformed into the corresponding outputs, and under which conditions (recall the discussion on the definition of pre- and post-conditions in section III).

D. Mapping to Realizing Components

While the primary focus of the reference architecture model is a decomposition of the MBEE system of interest from a functional perspective, our efforts include the specification of one or more illustrative reference

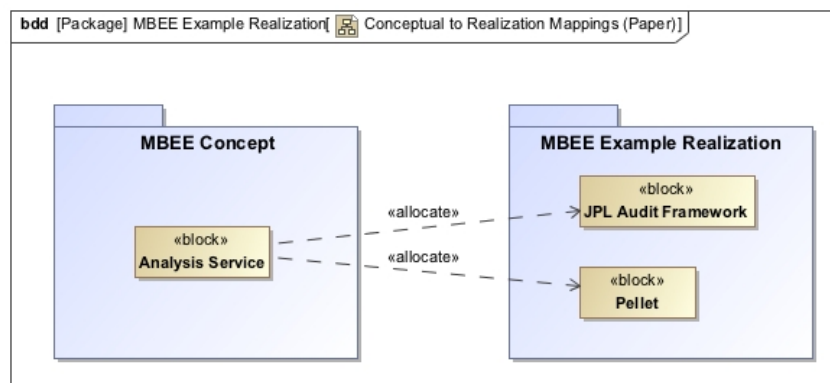


Figure 10. Example of function-to-realization mapping in the MBEE reference architecture model: within the context of a particular realization (here: the hypothetical “MBEE Example Realization”) the functionality of the conceptual “analysis service” is implemented (in part) by JPL’s in-house developed audit framework⁷ and the open source reasoning framework “Pellet”.⁸

realization specifications of (at least a subset of) the specified functionality. This specification of a number of *realization architectures* occurs in a similar manner to that of refining the functional architecture. In OOSEM, the fact that certain elements in the realization architecture (such as a particular SysML modeling tool) realize a particular set of specified functionality can be captured using SysML Allocation relationships. It should be noted that this is just one of several alternative ways in which a mapping between functional and realization architectures can be captured, an overview of which is outside the scope of this paper. Using SysML Allocations for defining a mapping from functional components to realizing components is illustrated in Figure 10. In the example, we allocate the functionality specified for the “Analysis Service” to two concrete software products: the open source OWL-DL⁹ reasoner “Pellet”,⁸ and a SPARQL-based model audit framework developed at JPL.⁷

Mappings between functional and realizing components are not always unique. Certain realizing components may only fulfill a subset of the functionality specified by the allocated functional component. In other situations, certain realizing components may fulfill functionality specified by multiple functional components. This means that a realization architecture can be structurally very different from the functional architecture.

V. Related Work

To the best of knowledge of the authors, there are no publications demonstrating efforts similar to our work on defining a unifying reference architecture for MBSE environments. However, there is a plethora of model integration frameworks and isolated model-integration efforts. There are also a number of enterprise-scale MBSE infrastructure frameworks that have been proposed. However, these have been developed specifically for use within certain application domains, or are tailored to organization-specific processes and applications.

Model integration frameworks include generic tool integration solutions such as ModelBus⁶ and Capella,¹⁰ but also optimization frameworks such as Phoenix Integration ModelCenter.¹¹ InterCAX’s Syndeia platform (previously SLIM)¹² is another example of a tool integration framework. AutoFocus3 is a complete model-driven environment for developing embedded systems utilizing models from requirements specification to hardware architecture.¹³ A number of isolated or limited model or tool integrations have also been reported, such as various implementations of the SysML - Modelica transformation,^{14,15} and tools capable of interfacing with external applications to simulate and analyze a system such as the Cameo Simulation Toolkit.¹⁶ The primary focus of most of these solutions is not the integration of information and knowledge, but rather the provision of bindings between tools given a specific interpretation of the underlying modeling language(s) and models.

A notable enterprise-scale MBSE framework called the “Digital Tapestry” initiative is in use and under continuing development at Lockheed Martin Corporation.¹⁷ Similarly, the System Architecture Virtual

Integration (SAVI) effort lead by a consortium of industry and academia has reported an architecture for multi-disciplinary, model-based development.¹⁸ Additionally, an enterprise framework focused on the automatic generation of gate products (primarily documents) from a SysML model has been reported on by JPL in the past.³

Most of these efforts have a seemingly similar objective. However, in all cases certain technological choices have been made that render most of these infrastructures incompatible with one another, while not covering the complete functionality required by a MBEE. The aim of the work presented in this paper is to define an underlying reference architecture which captures an implementation-neutral and technology-agnostic view on the functionality supported by the various efforts described in this section. It also offers an underlying framework for model integration upon which the realization of a MBSE environment can be based.

VI. Conclusion

This paper introduces and reports on the current status of JPL's efforts related to developing a reference architecture that serves as a specification for model-based engineering environments. The work is motivated by the need to unify the currently existing incompatible and disparate efforts aimed at implementing (parts of) MBSE environments. While the presented reference architecture is not complete, it already covers a broad spectrum of systems engineering activities and provides several interesting and valuable insights, summarized below.

A useful side effect of creating the reference architecture model has been the formal capturing of current systems engineering processes, practices, products and technologies, as well as the relationships between terminology used across various domains. This has provided the unique advantage of being able to identify areas of improvement to current processes, and how automation and formal modeling can aid in evolving these. It has also provided input to JPL's ongoing effort to formalize domains through ontologies.¹⁹ Both of these are informative both for the community as a whole and internally at JPL for the purpose of streamlining and improving processes.

The reference architecture model is developed in a scenario-driven fashion. Even though very specific systems engineering processes are considered as a primary source for driving use case scenarios, and the complete set of use cases is likely too large and complex to fully elaborate, it is expected that there is a closed set of functions that any implementation of a MBEE must support. We postulate that this closed set can be abstracted from the captured operational scenarios. For instance, the generation of a specific document can be generalized to the need for a document generation facility that works on specific types of inputs and exerts a certain behavior. To determine completeness of this set of core functionality, additional use cases should be modeled for purposes of validation. This core set of functionality should allow for any more specific (model-based) systems engineering processes to be realized. Future work will include identifying and capturing this core set of capabilities.

Acknowledgments

The work described in this paper was performed at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration. The authors would like to acknowledge the major ongoing contributions of Dr. Maged E. Elaasar. In addition, the authors would like to thank all current and past contributors and advisers to the reference architecture effort, particularly Christopher L. Delp, Alejandro Jimenez, Dr. Bjorn F. Cole, Sanford Friedenthal, Dr. J. Steven Jenkins, and Zachary Parent. Also acknowledged is the input provided by the internal reviewers of the reference architecture.

References

- ¹Cloutier, R., Muller, G., Verma, D., Nilchiani, R., Hole, E., and Bone, M., "The concept of reference architectures," *Systems Engineering*, Vol. 13, No. 1, 2010, pp. 14-27.
- ²OMG, "Systems Modeling Language (OMG SysML) 1.4 Specification," *Object Management Group, OMG Available Specification (September 2015)*.
- ³Delp, C., Lam, D., Fosse, E., and Lee, C.-Y., "Model based document and report generation for systems engineering," *Proceedings of the 2013 IEEE Aerospace Conference*, Big Sky, MT, 2013, pp. 1-11.
- ⁴Friedenthal, S., Moore, A., and Steiner, R., *A Practical Guide to SysML: the Systems Modeling Language*, Morgan

Kaufmann, 2014.

⁵Herzig, S. J., Qamar, A., Reichwein, A., and Paredis, C. J., “A conceptual framework for consistency management in model-based systems engineering,” *ASME 2011 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, American Society of Mechanical Engineers, 2011, pp. 1329–1339.

⁶Hein, C., Ritter, T., and Wagner, M., “Model-driven tool integration with modelbus,” *Workshop Future Trends of Model-Driven Development*, 2009, pp. 50–52.

⁷Cole, B. and Jenkins, J. S., “Connecting Requirements to Architecture and Analysis via Model-Based Systems Engineering,” *Proceedings of AIAA SciTech 2015 Forum*, 2015.

⁸Sirin, E., Parsia, B., Grau, B. C., Kalyanpur, A., and Katz, Y., “Pellet: A Practical OWL-DL Reasoner,” *Web Semantics: Science, Services and Agents on the World Wide Web*, Vol. 5, No. 2, 2007, pp. 51–53.

⁹McGuinness, D. L., Van Harmelen, F., et al., “OWL web ontology language overview,” *W3C recommendation*, Vol. 10, No. 10, 2004, pp. 2004.

¹⁰Blondelle, G., Bordeleau, F., and Exertier, D., “Polarsys: A New Collaborative Ecosystem for Open Source Solutions for Systems Engineering Driven by Major Industry Players,” *INCOSE INSIGHT*, Vol. 18, No. 2, 2015, pp. 35–38.

¹¹Malone, B. and Papay, M., “ModelCenter: an integration environment for simulation based design,” *Simulation Interoperability Workshop*, 1999.

¹²Bajaj, M., Zwemer, D., Peak, R., Phung, A., Scott, A. G., and Wilson, M., “SLIM: collaborative model-based systems engineering workspace for next-generation complex systems,” *Proceedings of the 2011 IEEE Aerospace Conference*, Big Sky, MT, 2011.

¹³Aravantinos, V., Voss, S., Teufl, S., Hölzl, F., and Schätz, B., “Autofocus 3: Tooling concepts for seamless, model-based development of embedded systems,” *Joint proceedings of ACES-MB*, 2015, pp. 19.

¹⁴Herzig, S. J., Rouquette, N. F., Forrest, S., and Jenkins, J. S., “Integrating analytical models with descriptive system models: implementation of the OMG SysML standard for the tool-specific case of MapleSim and MagicDraw,” *Procedia Computer Science*, Vol. 16, 2013, pp. 118–127.

¹⁵Paredis, C. J., Bernard, Y., Burkhart, R. M., Koning, H.-P., Friedenthal, S., Fritzson, P., Rouquette, N. F., and Schamai, W., “An Overview of the SysML-Modelica Transformation Specification,” *INCOSE International Symposium*, Vol. 20, Wiley Online Library, 2010, pp. 709–722.

¹⁶No Magic, Inc., “Cameo Simulation Toolkit,” 2011.

¹⁷Dean, M. A. and Phillips, M. J., “Model-based Advancements at Lockheed Martin Space Systems Company,” *AIAA SPACE 2015 Conference and Exposition*, 2015, p. 4461.

¹⁸Redman, D., Ward, D., Chilenski, J., and Pollari, G., “Virtual integration for improved system design,” *AVICPS 2010*, 2010, pp. 57.

¹⁹Jenkins, S., “Ontologies and Model-Based Systems Engineering,” *INCOSE International Workshop 2010 MBSE Workshop*, California Institute of Technology, Jet Propulsion Laboratory, Pasadena, CA, 2010.